

DAA Lab - Experiment 8

Implement Floyd–Warshall Algorithm for All-Pairs Shortest Path

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement Floyd–Warshall algorithm to find the shortest paths between all pairs of vertices in a graph.
- Understand how dynamic programming helps in solving shortest path problems efficiently.

- The Floyd–Warshall algorithm is a **Dynamic Programming** approach.
- It computes shortest paths between all pairs of vertices in a weighted graph.
- Works with positive and negative edge weights (but no negative cycles).
- Recurrence relation:

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$$

- Iteratively updates the distance matrix for each vertex as an intermediate node.

Algorithm: Floyd–Warshall

FloydWarshall(G, n)

- ① Initialize matrix *dist* with adjacency matrix.
- ② For $k = 0$ to $n-1$:
 - For $i = 0$ to $n-1$:
 - For $j = 0$ to $n-1$:
 - If $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$, update:

$$\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$$

- ③ Print the shortest distance matrix.

C Code - Floyd–Warshall Algorithm (Part 1)

```
#include <stdio.h>
#define INF 999

int main() {
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int dist[20][20];
    printf("Enter adjacency matrix (use 999 for INF):\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
}
```

C Code - Floyd-Warshall Algorithm (Part 2)

```
// Floyd Warshall Algorithm
for (int k=0; k<n; k++) {
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
        }
    }
}

printf("All-Pairs Shortest Path Matrix:\n");
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++) {
        if (dist[i][j] == INF)
            printf("INF\t");
        else
            printf("%d\t", dist[i][j]);
    }
    printf("\n");
}
```

C Code - Floyd–Warshall Algorithm (Part 3)

```
    return 0;  
}
```

Sample Input/Output

Input:

Number of vertices = 4

Adjacency matrix: 0 5 999 10

999 0 3 999

999 999 0 1

999 999 999 0

Output:

All-Pairs Shortest Path Matrix: 0 5 8 9

999 0 3 4

999 999 0 1

999 999 999 0

Complexity Analysis

- Time Complexity: $O(V^3)$
- Space Complexity: $O(V^2)$
- Works for negative weights but not for negative cycles.

Observation Table

Vertices	Execution Time (ms)
3	0.001
4	0.002
5	0.004
6	0.007

Comparison

- Dijkstra's algorithm computes shortest path from a single source.
- Floyd–Warshall finds shortest paths between all pairs of vertices.
- It is simpler but slower for large sparse graphs.

Viva Questions

- ① What is the key principle behind Floyd–Warshall algorithm?
- ② How does dynamic programming apply here?
- ③ Can the algorithm handle negative edge weights?
- ④ Why is the complexity $O(V^3)$?

Conclusion

- Floyd–Warshall algorithm efficiently finds shortest paths between all pairs of vertices.
- It uses dynamic programming and matrix updates iteratively.
- Suitable for dense graphs with manageable vertex counts.

References

- Cormen et al., *Introduction to Algorithms*.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.